Práctica: Configuración usando radio button y switch

1. Objetivo de la práctica

Desarrollar una aplicación móvil con Jetpack Compose que permita configurar las preferencias de un videojuego mediante controles interactivos, utilizando Switch, RadioButton, Column, Row, Text y estados con remember y mutableStateOf.

Al finalizar, el estudiante podrá:

- Utilizar un Switch para activar o desactivar una opción.
- Utilizar RadioButton para seleccionar una única opción entre varias.
- Gestionar estados para actualizar la interfaz automáticamente.
- Mostrar un resumen de las preferencias elegidas por el usuario.

2. Instrucciones:

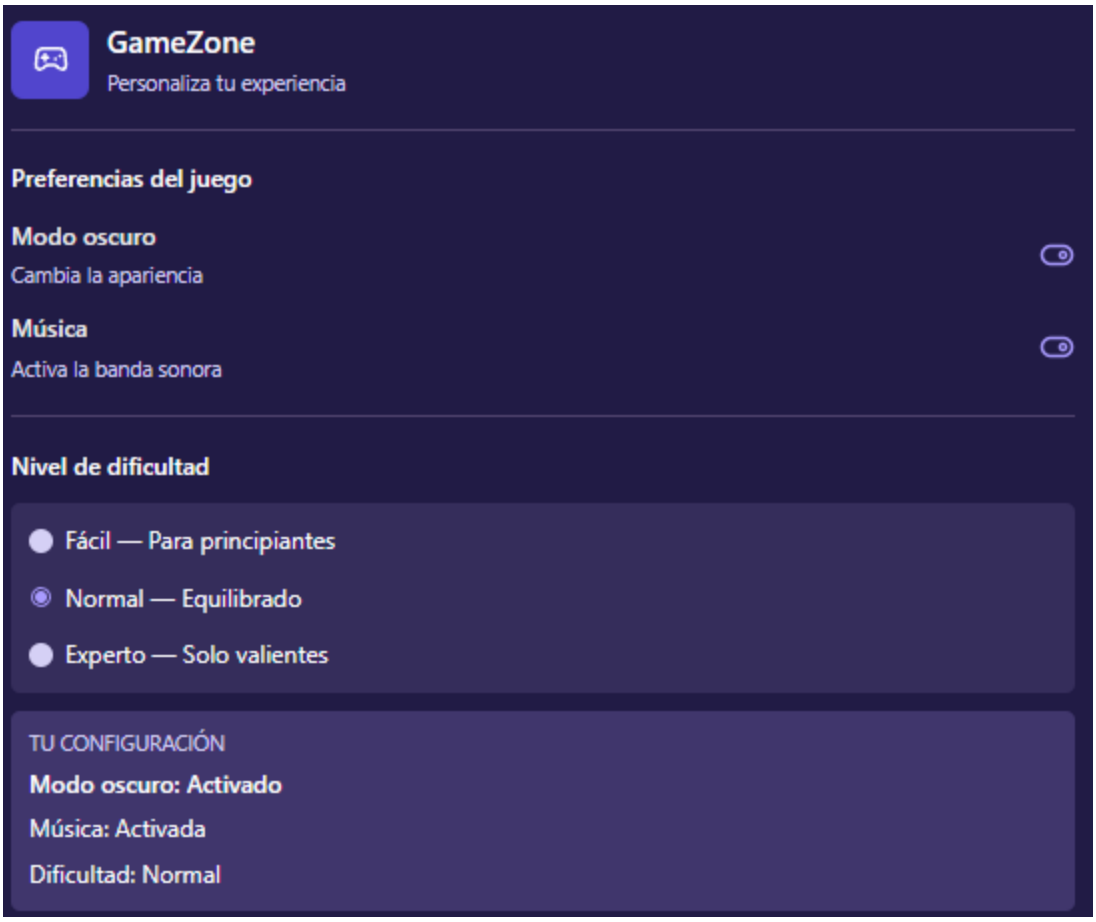
Imagina que estás desarrollando un videojuego móvil y necesitas una pantalla de configuración.

El usuario debe poder decidir:

- Modo oscuro: activar o desactivar la apariencia oscura.
- Música del juego: activar o desactivar la música.
- Dificultad: elegir entre Fácil, Normal o Experto.
- Resumen: visualizar la configuración seleccionada.

La aplicación deberá reaccionar inmediatamente a cada cambio, sin necesidad de presionar un botón para actualizar las preferencias.

3. Diseño esperado:



4. Código base:

```
package com.example.gamezone

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
```

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                GameZoneScreen()
            }
        }
    }
}

@Composable
fun GameZoneScreen() {

    // Estados de los Switch
    var modoOscuro by remember {
        mutableStateOf(false)
    }

    var musica by remember {
        mutableStateOf(true)
    }

    // Estado del RadioButton
    var dificultad by remember {
        mutableStateOf("Normal")
    }

    Surface(
        modifier = Modifier.fillMaxSize(),
        color = if (modoOscuro)
```

```
            MaterialTheme.colorScheme.surface
        else
            MaterialTheme.colorScheme.background
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(24.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {

            Text(
                text = "🎮 GameZone",
                style = MaterialTheme.typography.headlineMedium
            )

            Text("Personaliza tu videojuego")

            HorizontalDivider()

            Text(
                text = "Preferencias",
                style = MaterialTheme.typography.titleLarge
            )

            // SWITCH 1
            Row(
                modifier = Modifier.fillMaxWidth(),
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(
```

```
        text = "Modo oscuro",
        modifier = Modifier.weight(1f)
    )

    Switch(
        checked = modoOscuro,
        onCheckedChange = {
            modoOscuro = it
        }
    )
}

// SWITCH 2
Row(
    modifier = Modifier.fillMaxWidth(),
    verticalAlignment = Alignment.CenterVertically
) {
    Text(
        text = "Música del juego",
        modifier = Modifier.weight(1f)
    )

    Switch(
        checked = musica,
        onCheckedChange = {
            musica = it
        }
    )
}

HorizontalDivider()
```

```
Text(
    text = "Dificultad",
    style = MaterialTheme.typography.titleLarge
)

val niveles = listOf(
    "Fácil",
    "Normal",
    "Experto"
)

niveles.forEach { nivel ->
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .heightIn(min = 48.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        RadioButton(
            selected = dificultad == nivel,
            onClick = {
                dificultad = nivel
            }
        )

        Text(
            text = nivel,
            modifier = Modifier.padding(start = 8.dp)
        )
    }
```

```
}

HorizontalDivider()

Text(
    text = " 📋 Tu configuración",
    style = MaterialTheme.typography.titleLarge
)

Text(
    text = "Modo oscuro: ${
        if (modoOscuro) "Activado" else "Desactivado"
    }"
)

Text(
    text = "Música: ${
        if (musica) "Activada" else "Desactivada"
    }"
)

Text("Dificultad: $dificultad")

Spacer(modifier = Modifier.height(8.dp))

Button(
    onClick = {
        // Aquí puedes agregar una acción extra.
    },
    modifier = Modifier.fillMaxWidth()
) {
```

```
            Text("¡Listo para jugar!")
        }
    }
  }
}
```

Explicación:

Siwtch: Un Switch permite activar o desactivar una opción. Por ejemplo, encender o apagar la música.

```
var musica by remember {
  mutableStateOf(true)
}

Switch(
  checked = musica,
  onCheckedChange = { nuevoValor ->
    musica = nuevoValor
  }
)
)
```

- remember conserva el estado durante las recomposiciones.
- mutableStateOf(true) inicia la música como activada.
- checked indica si el interruptor está encendido.
- onCheckedChange recibe el nuevo valor cuando el usuario cambia el interruptor.

Así funciona un RadioButton: Permite seleccionar una sola opción dentro de un grupo.

```
var dificultad by remember {
  mutableStateOf("Normal")
}
```

```
RadioButton(
    selected = dificultad == "Experto",
    onClick = {
        dificultad = "Experto"
    }
)
```

Si el usuario selecciona Experto, la variable dificultad cambia y la interfaz actualiza cuál opción aparece seleccionada.
La clave es que todos los botones de dificultad compartan la misma variable de estado.